

Data Acquisition -Web Scrapping

Statistics-Index Hierarchy Data Acquisition: Gather information from different sources, Web APIs, Open Data Sources, Web Scrapping

Data Acquisition- Web Scrapping

Hierarchical Indexing

More common pattern in practice is to make **use of *hierarchical indexing*** (also known as ***multi-indexing***) to incorporate multiple index ***levels*** within a single index.

```
In[2]: index = [('California', 2000), ('California', 2010),
                ('New York', 2000), ('New York', 2010),
                ('Texas', 2000), ('Texas', 2010)]
populations = [33871648, 37253956,
               18976457, 19378102,
               20851820, 25145561]
pop = pd.Series(populations, index=index)
pop
```

```
Out[2]: (California, 2000)    33871648
        (California, 2010)    37253956
        (New York, 2000)     18976457
        (New York, 2010)     19378102
        (Texas, 2000)        20851820
```

Data Acquisition Web Scrapping

But the convenience ends there. For example, if you need to select all values from 2010, you'll need to do some messy (and potentially slow) munging to make it happen:

```
In[4]: pop[[i for i in pop.index if i[1] == 2010]]
```

```
Out[4]: (California, 2010)    37253956  
        (New York, 2010)     19378102  
        (Texas, 2010)        25145561  
        dtype: int64
```

MultiIndex contains multiple *levels* of indexing—in this case, the state names and the years, as well as multiple *labels* for each data point which encode these levels.

Data Acquisition Web Scrapping

Pandas MultiIndex

Pandas MultiIndex type gives us the type of operations

```
In[5]: index = pd.MultiIndex.from_tuples(index)
       index
```

```
Out[5]: MultiIndex(levels=[['California', 'New York', 'Texas'], [2000, 2010]],
                    labels=[[0, 0, 1, 1, 2, 2], [0, 1, 0, 1, 0, 1]])
```

Data Acquisition Web Scrapping

If we reindex our series with this MultiIndex, we see the hierarchical representation of the data:

```
In[6]: pop = pop.reindex(index)
      pop
```

Out[6]: California	2000	33871648
	2010	37253956
New York	2000	18976457
	2010	19378102

Here the first two columns of the Series representation show the multiple index values, while the third column shows the data. Notice that some entries are missing in the first column: in this multi-index representation, any blank entry indicates the same value as the line above it.

Data Acquisition Web Scrapping

Now to access all data for which the second index is 2010, we can simply use the Pandas slicing notation:

```
In[7]: pop[:, 2010]
```

```
Out[7]: California    37253956  
        New York      19378102  
        Texas         25145561  
        dtype: int64
```

WEB API

An **API (Application Programming Interface)** is the definition of the way computer programs communicate with each other. We use Requests to connect to the API of a web server, tell it what we want, and it returns it to us. This is called the **request-response** cycle.

Requests is a Python library that makes **requests** to web servers. It provides a nice interface and is one of the go-to tools. It does the raw data-download for simple web servers.

WEB API

Let's make a request to the Cat Fact API. If we go to <https://catfact.ninja/>, it gives us the definitions:

- **GET** `/fact` is the **API endpoint**.
- **GET** is the type of request we make and `/fact` is the **path**.

You can even test this in your web browser: <https://catfact.ninja/fact>

Using the Requests library, we do this with `get()`.

Web API

```
# Import
```

```
import requests
```

```
# URL
```

```
url = 'https://catfact.ninja/fact'
```

```
# Make a request
```

```
response = requests.get(url)
```

```
response_content = response.content
```

```
# Display
```

```
display(response_content)
```

The [`requests.Response`](#) object tells us what the server said. We can access the response content using [`content`](#).

The response content is in the [`JSON format`](#) and Requests gives us the [`json\(\)`](#) method that decodes it and returns the corresponding data as Python objects. This is equivalent to [`json.load\(\)`](#).

Web API

```
response_json = response.json()  
# Display display(response_json)
```

Output:

```
{'fact': 'Ginger tabby cats can have freckles around their mouths and on  
their eyelids!', 'length': 77}
```

Web Scrapping

```
# URL
url = 'http://universities.hipolabs.com/search'

# Make the parameter dictionary
parameters = {'country' : 'Finland'}

# Get response
response = requests.get(url,
params=parameters)

# Decode JSON
response_json = response.json()

# Display
display(response_json[:2])
```

Output:

```
[{'state-province': None,
'alpha_two_code': 'FI', 'country':
'Finland', 'web_pages': [http://
www.abo.fi/], 'domains': ['abo.fi'],
'name': 'Abo Akademi University'},
{'state-province': None,
'alpha_two_code': 'FI', 'country':
'Finland', 'web_pages': [http://
www.cou.fi/], 'domains': ['cou.fi'],
'name': 'Central Ostrobothnia
University of Applied Sciences'}]
```

Web Scrapping

Web scraping is a technique to automatically access and extract large amounts of information from a website, which can save a huge amount of time and effort.

Process of Web Scrapping

- 1.Importing the required libraries
- 2.Specifying the URL containing the dataset and passing it to `requests.get()` to get the HTML content of the page.
 - 1.Using BeautifulSoup to parse the HTML content
 - 2.Extracting the required information from the data
- 3.Saving the pandas dataframe as a CSV file called **Amazon Data.csv**

Web Scrapping

Web Scraping Workflow

The workflow for web scraping with Python can be divided into the following three steps:

Obtaining the HTML: Firstly, we need to send an HTTP request to the web page server that we want to scrape. If the request is successful, the server will respond with the HTML content of the page.

Parsing the HTML: Most of the obtained HTML data is nested, making it difficult to extract information using standard string processing techniques. Instead, we need a parser, i.e. an algorithm designed to parse the HTML and create a parse/syntax tree of the HTML data.

Web Scrapping

Extracting the Data: Once the syntax tree is created, we need to navigate it and retrieve the information that we are interested in

To complete those steps, we need two third-party Python libraries:

Requests: a simple but powerful library for sending all kinds of HTTP requests to a web server,

Beautiful Soup: a library for parsing HTML and XML documents. It works with a user-selected parser to provide idiomatic ways of navigating, searching, and modifying the parse tree.

Open Data Sources

open data sources for data science include:

Kaggle (a large repository of diverse datasets), Github, UCI Machine Learning Depository

Google Dataset Search (a search engine for publicly available datasets), Academic Torrents Data

[AWS Datasets](#), [unicef datasets](#)

World Bank Open Data (global development data),

US Census Bureau (population, economy, and health data),

AWS Public Datasets (datasets from Amazon Web Services),

NASA Earth Observations (climate and environmental data), and

the EU Open Data Portal (broad range of European data).